
SUB-EVENT DETECTION – KAGGLE CHALLENGE

L.E.S. IS MORE

Lara Hofman

Sandro Mikautadze

Elio Samaha

Academic Year 2024/25 - École Polytechnique

1 Introduction

We address the challenge of predicting sub-events in tweets. There is a huge volume of social interactions during major worldwide events, therefore predicting the presence of sub-events from tweets can be a promising application. More precisely, our task is to create a binary classifier to detect the presence or absence of sub-events in minute-level periods during football games. The data set is publicly available on Kaggle [1] and contains matches from the 2010 and 2014 World Cups. The evaluation metric is accuracy.

In this report, we outline our methodology, beginning with data processing and feature engineering (Section 2). We then detail our model architecture and training strategies (Section 3). Results and potential improvements are discussed in Section 4, with additional technical details and alternative approaches (not included in the final Kaggle submission) in the appendix.

2 Data

The dataset consists of approximately six million tweets, with five million allocated for training (from 16 games) and one million for testing (from 4 games). Each tweet included an identifier for the corresponding football match, an identifier for the one-minute time period within the match, a Unix timestamp indicating the exact time the tweet was posted, and a binary label indicating the presence (1) or absence (0) of a sub-event during that time period. Data preprocessing was a crucial step to ensure a more streamlined and less noisy data set. This process involved:

- **Cleaning and Normalization:** Duplicate tweets and retweets were removed. Text was converted to lowercase, additional symbols such as hashtags (#) and newline characters (\n) were eliminated, user mentions and URLs were replaced with placeholders ('@user', 'HTTPURL') and emojis were converted into their text string representations. In addition, references to teams were replaced with generic placeholders such as <team1>, <team2>, or <team> when discussing other teams that did not play in that specific game. After this cleaning process, the dataset was reduced to nearly half its original size.
- **Tokenization:** Bertweet [2] tokenizer was used, with a token limit of 50 per tweet to optimize memory efficiency while retaining essential information. Indeed, analysis of the train and test sets revealed that the 99th percentile of token lengths were 52 and 53, respectively, while the maximum token lengths were 257 and 255. By capping tokenization at 50 tokens, we likely removed information from some tweets but gained considerable efficiency in processing speed.
- **Embedding Generation:** To generate a single representation for each period of every match, each tweet was embedded using Bertweet, and the [CLS] token from the embeddings was extracted to represent the overall context of the tweet. These embeddings were aggregated over periods using a similarity-based approach. The mathematical details of this aggregation are included in the appendix. This process produced a single vector of dimension 768 for each period within each match.
- **Dimensionality Reduction:** While these embeddings provided a meaningful representation, visualizations using t-SNE and UMAP showed insufficient separability between embeddings. To address this, we projected the embeddings into a lower-dimensional space of size 256 using supervised contrastive loss. This approach further refined the embeddings, improving their quality and separability.

To further improve predictive power, feature engineering added metrics such as Hashtags Count, Exclamation Count, Tweet Length and temporal features like ‘Tweet Count Per Period’, PreviousN Tweet Frequency, and TopTFIDFWordCount. We observed slight differences in the evolution and distribution of features based on the teams playing—whether two equally matched teams or one team significantly stronger than the other. To account for this, we assigned scores to each team based on their performances during the years 2010-2014. A new feature, Team_Score, was introduced, calculated as the absolute difference between the scores of the two teams in a match. These features were merged with the reduced embeddings to create an enriched dataset for modeling.

3 Modeling Pipeline

In this section, we present the modeling pipeline used for the challenge. The goal of this pipeline is to systematically develop, refine, and evaluate the model before producing a final submission.

3.1 Initial Model Selection and Baseline Evaluation

Our modeling approach evolved incrementally, starting with simpler architectures and progressing to more sophisticated designs. Initially, we used traditional machine learning models such as Random Forest, Decision Tree, SVM, XgBoost with Glove embeddings given in the baseline and no additional features.

To be able to capture the temporal aspect of our data, we decided to develop a sophisticated model architecture combining Long Short-Term Memory (LSTM) networks with attention mechanisms, residual connections, and batch normalization. This combination was selected to effectively capture the temporal dependencies and contextual relevance of tweets, ensuring accurate identification of significant sub-events. This was inspired by methodologies such as those proposed by Bekoulis et al. [3], which treat sub-event detection as a sequence labelling problem.

In parallel, we extended our feature engineering (Section 2) to include statistical and semantic features derived from tweet metadata and text. We trained tree-based models, including Random Forest, CatBoost, and XGBoost, on the enriched feature set combined with reduced Bertweet embeddings. Since the original embeddings were 256-dimensional and the dataset included 12 additional features, we reduced the embeddings to 25 dimensions using Principal Component Analysis (PCA) to ensure the features maintained their importance relative to the embeddings. We were able to keep 99% of the variance using only 25 components. This resulted in a combined 37-dimensional feature space. We initially focus on the Random Forest model due to its robustness and interpretability. We randomly select a set of hyperparameters and train the RF model on a subset of the training data. Without performing cross-validation at this stage, we use a simple train-test split (or a small sample) to quickly evaluate model performance. The objective is to verify that the model can achieve a baseline accuracy that justifies further tuning. Achieving a reasonable accuracy here confirms that the Random Forest can capture meaningful patterns in the data, encouraging us to proceed.

3.2 Hyperparameter Tuning via Grid Search

Having established a viable baseline, we proceed to fine-tune the Random Forest hyperparameters. This step employs a grid search procedure that explores combinations of critical parameters, such as: `n_estimators` (the number of trees in the forest), `max_depth` (the maximum depth of each tree), `min_samples_split` and `min_samples_leaf` (parameters controlling tree complexity and regularization). For each parameter combination, we train a Random Forest model and evaluate its performance. The search aims to identify a parameter set that yields the highest validation accuracy, thus balancing complexity and generalization.

3.3 Cross-Validation for Robust Evaluation

To obtain a reliable estimate of the model’s performance and ensure the chosen hyperparameters generalize well beyond a single split, we employ K-fold cross-validation. Instead of relying on one fixed train-test partition, the dataset is split into multiple folds (e.g., five), and we iteratively train on $k - 1$ folds while validating on the remaining fold. This process is repeated for each fold, and the validation accuracies are averaged. Cross-validation reduces the risk of overfitting to a particular subset of the data and provides a more stable and trustworthy estimate of how well the model might perform on unseen samples.

3.4 Final Training and Submission

After identifying the best hyperparameter configuration and confirming its stability through cross-validation, we retrain the Random Forest model using the entire training dataset. This step leverages all available data to maximize the model’s

predictive power. Finally, the fully trained and optimized Random Forest model is applied to the test dataset. The resulting predictions are formatted into a submission file as required by the competition guidelines. By following this pipeline, we ensure that our final submission is both well-validated and robust, having undergone baseline evaluation, fine-tuning, and thorough cross-validation before final deployment. The results are available in the next section.

4 Results and Ensemble Approach

4.1 Individual Model Performance

The Random Forest model utilizing both engineered features and reduced Bertweet embeddings emerged as the best-performing model with a validation score of 0.76562. The LSTM model also performed well, scoring 0.72265. However, the ensemble approach combining both models did not yield the expected improvements, achieving only 0.57812, likely due to redundancy in input information.

4.2 Ensemble Model

We attempted a stacking-based ensemble combining LSTM and Random Forest models. Probabilities predicted by each model for event type 1 were used as inputs to a logistic regression model, with a threshold of 0.5 for classification. Despite rigorous hyperparameter tuning, the ensemble model achieved a disappointing score of 0.57812 on Kaggle evaluation data. This underperformance was likely due to redundancy, as both models relied heavily on embeddings, reducing the diversity of the ensemble’s inputs.

Model Configuration	Validation Score
Random Forest (No Features, Glove Embeddings)	0.66015
LSTM	0.72265
Random Forest (Features + Reduced Bertweet Embeddings)	0.76562
Ensemble (LSTM + Random Forest)	0.57812

Table 1: Performance of different models on validation data.

5 Limitations and Future Work

5.1 Limitations

Several challenges limited the performance and generalizability of our models: the dataset size was relatively small, comprising only 20 games, which constrained the training of more complex models, the noisy nature of tweets and weak correlation between features and event types introduced significant variability, making sub-event detection more challenging, a lack of additional contextual features, such as the tournament phase (e.g., group stage vs. finals), further limited the models’ ability to differentiate between sub-events effectively.

To address these limitations, we propose the following directions for future research: train Random Forest exclusively on engineered features, excluding embeddings, to create a diverse ensemble with LSTM and reduce redundancy, augment the dataset or incorporate external data sources, such as sentiment scores or live commentary, to improve model robustness and introduce contextual features, such as the stage of the tournament, to provide additional information for better sub-event detection.

References

- [1] Christos.X. Kaggle challenge. <https://www.kaggle.com/competitions/sub-event-detection-in-twitter-streams/overview> [Accessed: 12/12/2024].
- [2] Dat Quoc Nguyen, Thanh Vu, and Anh Tuan Nguyen. Bertweet: A pre-trained language model for english tweets, 2020.
- [3] Giannis Bekoulis, Johannes Deleu, Thomas Demeester, and Chris Develder. Sub-event detection from twitter streams as a sequence labeling problem. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence*, pages 2620–2627. AAAI Press, 2017.

Appendix

Embedding Aggregation and Contrastive Learning

To generate a single embedding for each match period, we aggregated the individual tweet embeddings using a similarity-based weighting mechanism. Given the CLS embeddings of tweets for a specific period $\mathbf{E} = [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n]$ where $\mathbf{e}_i \in \mathbb{R}^{768}$, the process is as follows:

1. Reference Embedding: Compute the mean embedding $\mathbf{r} = \frac{1}{n} \sum_{i=1}^n \mathbf{e}_i$ as a reference vector.
2. Cosine Similarity Weights: For each tweet embedding $\mathbf{e}_i$, calculate the similarity score with the reference embedding:

$$\text{sim}(\mathbf{e}_i, \mathbf{r}) = \frac{\mathbf{e}_i \cdot \mathbf{r}}{\|\mathbf{e}_i\| \|\mathbf{r}\|}.$$

Normalize these scores using a softmax function to generate attention weights:

$$w_i = \frac{\exp(\text{sim}(\mathbf{e}_i, \mathbf{r}))}{\sum_{j=1}^n \exp(\text{sim}(\mathbf{e}_j, \mathbf{r}))}.$$

3. Weighted Aggregation: Compute the aggregated embedding for the period as:

$$\mathbf{E}_{\text{agg}} = \sum_{i=1}^n w_i \mathbf{e}_i.$$

4. Contrastive Projection: The aggregated embeddings $\mathbf{E}_{\text{agg}}$ were further refined using a projection head, $\mathbf{z} = f_{\text{proj}}(\mathbf{E}_{\text{agg}})$, to map them into a lower-dimensional space suitable for contrastive learning. Here, f_{proj} is a two-layer neural network:

$$\mathbf{z} = \sigma(\mathbf{W}_2 \text{PRELU}(\mathbf{W}_1 \mathbf{E}_{\text{agg}} + \mathbf{b}_1) + \mathbf{b}_2),$$

where $\mathbf{W}_1, \mathbf{W}_2, \mathbf{b}_1, \mathbf{b}_2$ are learnable parameters.

5. Supervised Contrastive Loss: To ensure that embeddings of periods with the same event type were closer in the embedding space, we used a supervised contrastive loss. For embeddings $\mathbf{z}_i$ and labels y_i , the loss was defined as:

$$\mathcal{L}_{\text{contrastive}} = -\frac{1}{N} \sum_{i=1}^N \frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_p / \tau)}{\sum_{j=1}^N \exp(\mathbf{z}_i \cdot \mathbf{z}_j / \tau)},$$

where $P(i)$ is the set of positive pairs for i , τ is the temperature parameter, and $\cdot$ denotes the dot product between normalized embeddings.

This method ensured the creation of robust embeddings that both preserved the semantic meaning of the tweets and provided meaningful separation between periods with and without sub-events.